

DESIGN AUDIT — MULTI-AGENT
REVISION PIPELINE

NBT Round Pipeline Design Audit

A static, deep-dive review of the scoring, judging and convergence design: cross-agent criteria conflicts, score non-convergence, judge cost structure, and the pricing of grammar and semantic defects.

Static analysis · commit 7d19d80 (2026-09-22)
Evidence cited as file:line · 15 findings, 15 fixes

PREPARED FOR THE PIPELINE OPERATOR · SEPT 2026

Table of Contents

Executive Summary	2
The System Under Audit	4
The judge scoring contract	5
Cross-Agent Criteria Conflicts (Hypothesis 1)	6
D-01 - The writing rubric swallows semantic defects	6
D-02 - The 'cannot drift' vocabulary drifted	7
D-03 - Em-dash: one rule, four meanings	7
D-04 - The judge is promised differences the normalizer deletes	8
Convergence Mechanics (Hypotheses 2 and 3)	9
D-07 - The stasis ratchet: 0-expected panels, asymmetric scrutiny, incumbent retention	9
D-08 - Too much scoring: the judge redundancy	9
D-09 - The merge-into-a-base strategy is suboptimal, and it is the only accumulator	10
D-10 - Too much rewriting at the beginning; paraphrase laundering across rounds	10
D-11 - All-or-nothing panels can void a round	11
Judge vs. Agent Perception Mismatch (Hypothesis 4)	11
D-12 - Self-reported tie-breaks are the only agent voice, and they are unverifiable	12
D-13 - Arithmetic overrides judgment	12
Why Grammar and Semantics Cannot Win (Hypothesis 5)	13
Additional Findings (Open-Ended Sweep)	14
D-14 - Judge grounding is warning-only	14
D-15 - Operational defaults and miscellany	14
Root-Cause Synthesis and Recommendations	15
Recommendations	15
Appendix A - Evidence Index	17
Appendix B - Verification of the Original Hypotheses	18

Executive Summary

This report is a static, deep-dive design audit of the NBT round-based revision pipeline (commit **7d19d80**, "Add round-scoped --only, per-round integrator masks and judge counts", imported 2026-09-22). The pipeline drives a population of codex agents over a Nature Biotechnology manuscript package: every round it stages one base copy, M rewrites, one frozen review, N revisions of that review, $K = 1 + M + N$ integrations, and a blind panel of judge sessions; a deterministic ranking then pins a champion that becomes the next round's base. The audit traced every prompt template, the judge scoring contract, the aggregation and champion-selection code, the OOXML normalizer, and the repo's own four audit ledgers, and verified each of the five issue categories you suspected. Two of them are confirmed in an unexpected form, one is confirmed and then some, one is partially refuted, and one is structurally guaranteed by the scoring arithmetic.

The headline result is that the pipeline is not one scoring function but **at least four that only partially overlap**: the reviewer's disposition bars, the auditor's promotion bar, the reviser's mandatory work list, and the judge's tier-capped arithmetic each price the same defect differently. The em-dash rule is the cleanest example: it is an advisory row for the reviewer and invisible to the auditor (it is not in FINDING_TIER_RULES), mandatory work for the reviser (M20_REWISE_RULE), at most a one-point formatting nudge for the judge, and the tooling's own nested-parenthesis remediation text recommends introducing an em dash while the density cap penalizes it. Your specific hypothesis that "the reviser favors em-dashes" is not true in this tree; what is true is subtler and arguably worse: revisers are **obligated** to spend effort on a defect class the panel can reward by at most 1 point out of a 9-point scale, so the effort is structurally invisible to selection.

On convergence: scores cannot converge across rounds because nothing accumulates. Each round's median is a field-relative measurement with fresh sampling noise over $2 * \text{judges} * (|\text{field}| - 1)$ integer scores, real effect sizes between successive candidates are around one rung on a -4..+4 scale, the incumbent retention rule freezes any statistical tie to the status quo, and the only cross-round signals (the vs-original gate and the prior-round finding reconciliation) are either coarse (six scores, and it stops binding after round 1) or location-based (a rewrite that paraphrases a defective sentence makes the carried finding "not reproducible" and the defect escapes the ledger). The cost side is worse: the default plan spends roughly 29 agent sessions in round 1, of which 21 are judge sessions that each re-run the entire frozen sweep set and produce ~26 disposition cells per comparison, with three judges per candidate triplicating identical sweep work that the orchestrator itself already computes and then throws away for blinding reasons.

ID	Finding	Severity	Primary evidence
D-01	Judge writing rubric routes semantic defects (Q1 premise, Q2 logic slip) into the minor-only, +/-1-capped writing tier; the same defect could be correctness (+/-4) depending on the judge's classification	High	nbt_pipeline.py:11551-11572, 1404-1413, 3873-3878
D-04	Judge sweep block promises un-normalized spelling/hyphenation consistency rows; the orchestrator auto-normalizes them away before judging, and per-package 'dominant' normalization can create new cross-package inconsistency	High	nbt_pipeline.py:1872-1885 vs nbt_docx_format.py:56-76, 1931-1949; nbt_pipeline.py:6141-6191

ID	Finding	Severity	Primary evidence
D-06	No cross-round comparable metric; field-relative medians re-measured with fresh noise, effects near one rung; nothing accumulates, so round outcomes are noise-dominated	High	nbt_pipeline.py:12826-12990, 13307-13309
D-10	Rewrites run before any review exists; prior-finding reconciliation is quote/location-based, so paraphrasing rewrites launder defects out of the carried list	High	nbt_pipeline.py:3417-3419, 10738-10755
D-08	Judge redundancy: full sweep re-run per judge session, ~26 check dispositions per comparison, 3x triplicated per candidate; deterministic code-side scans are recomputed by the orchestrator and discarded	High (cost)	nbt_pipeline.py:3769-3784, 10438-10440, 12690-12712
D-09	$K = 1+M+N$ full integrations per round are conservative and base-biased; the only cross-arm accumulation channel systematically under-absorbs losing arms' improvements and splits panel votes	Med-High	nbt_pipeline.py:3240-3260, 184-187
D-02	Tier vocabulary drift: judge prompt priority order lists 5 tiers, BASIS_TIERS has 6, DEFECT_CLASS_RULE says 'Five classes' and renders 6	Medium	nbt_pipeline.py:3899-3901, 1399-1400, 2061, 2101
D-03	Em-dash: advisory for reviewer/auditor, mandatory for reviser, ≤ 1 point for judge, and FMT-T8b fix text suggests adding em dashes	Medium	nbt_docx_format.py:1639-1642, 2155-2161, 595-607; nbt_pipeline.py:1849-1860, 1872-1885
D-05	Auditor adds (AU-*) are unverified and unchallengeable; auditor drops need strong evidence; reviser can still discard confirmed findings at R1; audit off by default	Medium	nbt_pipeline.py:3092-3197, 2896-2899, 4371-4378
D-07	0-expected panels + deep-scrutiny asymmetry + incumbent retention ratchet weaken the selection gradient	Medium	nbt_pipeline.py:3846-3851, 3769-3783, 13312-13324
D-11	Panel completeness is all-or-nothing; one missing comparison can discard a whole round's fresh work via the base fallback	Medium	nbt_pipeline.py:13231-13264, 3974-3984
D-12	Self-reported critical_remaining/writing_remaining are ranking keys, unverified for rewrite/integrate arms; manual_steps excluded for the same incentive	Medium	nbt_pipeline.py:13112-13140, 13305-13309
D-13	Judge integer must equal the capped ledger sum; arithmetic overrides judgment and pressures row inflation	Low-Med	nbt_pipeline.py:12402-12430, 12558
D-14	Judge grounding (M18/M19/M20 artifacts) enforced as warning-only	Low-Med	nbt_pipeline.py:12690-12712
D-15	Ops defaults: 255 concurrent agent sessions, 4h per-run timeout, no cost governor	Low	nbt_pipeline.py:757-770

Table 1 - Top findings and severity

Chapters 2 through 6 verify your five suspected categories in order, chapter 7 adds the open-ended sweep results, and chapter 8 synthesizes the root cause: the pipeline is a conservative selector, not an optimizer, and revision effort is mispriced relative to panel reward. Appendix A indexes every citation, and Appendix B gives a verdict table for each of your original hypotheses. Every fix in chapter 8 is deliberately brief - one or two sentences - as requested; none of them requires re-architecting the pipeline, and most are prompt-constant or constant-table changes.

The System Under Audit

Each round r stages a fixed plan: a base copy $a1$ (round 1 is the pristine submission, later rounds are the previous champion), M rewrite arms with alternating declared levels (odd arms structural, even arms sentence-level), one frozen review, N revise arms that consume that frozen review, and $K = 1+M+N$ integration arms, one per pool member (nbt_pipeline.py:147-159, 172-195). The round's field is {original} plus every earlier pin plus the fresh arms plus the integrations, deduplicated by content digest - so the field grows with r and the pristine original never leaves the field (nbt_pipeline.py:146-153). With the default plan (rounds=2, judges=3, rewrites=[2,1], revises=[1,1], all integrators, nbt_pipeline.py:757-770) round 1 fields 7 members and round 2 fields 7 again with one fewer rewrite but an extra pin. A third round would field 8.

Judging is one session per (candidate, judge index): each of the 3 judges for candidate V runs the frozen sweep set on V 's blinded view, reads every opponent in full, and writes one directed integer per opponent plus a graded ledger. Per version that yields judges x ($|field|-1$) own scores plus the negated received scores, i.e. $2*judges*(|field|-1)$ directed scores - 36 at the default 7-member field (nbt_pipeline.py:12919-12924, README.md:129-168). The champion ranking key is (-median, -mean, IQR, critical_remaining, writing_remaining, content digest, id) over the flat directed-score list, with an incumbent-retention rule on exact statistical ties (nbt_pipeline.py:13307-13324). Fresh arms must additionally clear the anti-regression gate: median vs-original ≥ 0 over 6 scores (nbt_pipeline.py:2453-2462, 12983-12984).

Stage	Arms	What it does	Sessions (r1 / r2)
a1	1	Base copy only, no agent; round 1 = pristine copy, later = previous champion	0 / 0
rewrite	M	Full alternative versions; odd arms structural, even arms sentence-level; runs FIRST, before the review	2 / 1
review	1	One frozen identification pass (nbt-review M1-M24 + J1-J4 + discovery round)	1 / 1
audit	0/1	Optional independent auditor: confirm/drop every frozen finding, promote boilerplate OK closures to AU-* findings	off by default
revise	N	Reviewed-and-revised candidates consuming the frozen (or audited) finding list	1 / 1
integrate	$K=1+M+N$	Every pool member reworked with the WHOLE pool as donors; DIFF_LEDGER with per-row artifacts	4 / 3
judge	$judges \times field $	Blind pairwise panels: full frozen sweep on target + full read of every opponent + scores.json ledger	21 / 21
total	-	Agent sessions launched per round	29 / 27

Table 2 - Stages and default session counts per round

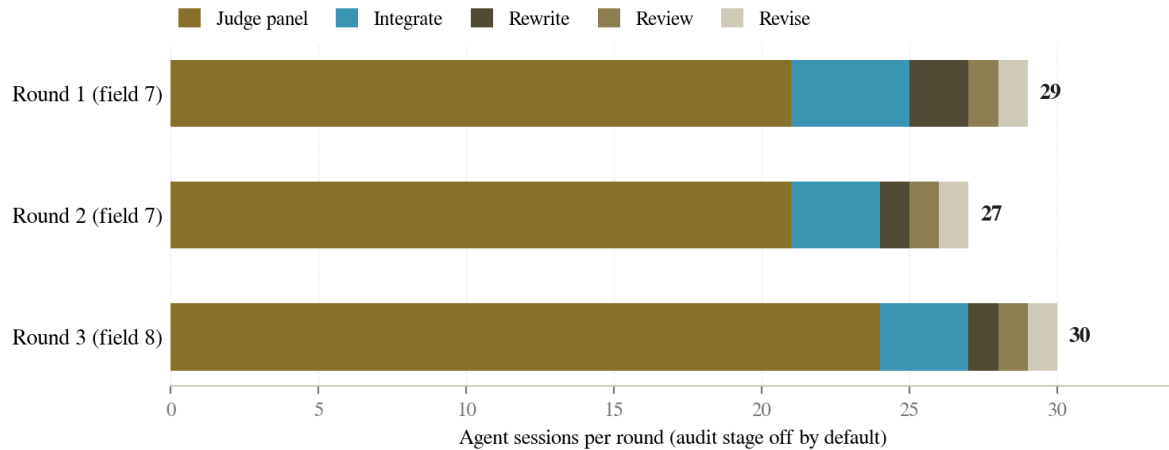


Figure 1 - Agent sessions per round under the default plan (judges=3; audit off; a 3rd round would run 30 sessions, of which 24 are judges).

The judge scoring contract

The judge prompt (JUDGE_DIRECTIVES, nbt_pipeline.py:3729-4003) is a dense machine-checkable contract. The judge scores one target against every opponent, one direction only, on a fixed integer scale of -4..+4; the integer must equal the capped sum of defect-ledger rows the judge enumerates (weights: minor 1, major 2, critical 3; per-tier caps: correctness +/-4, consistency +/-3, preservation +/-3, completeness +/-2, formatting +/-1, writing +/-1; formatting and writing admit minor rows only, so neither can ever be decisive; nbt_pipeline.py:3853-3895, 1404-1413). The orchestrator re-derives the integer from the ledger and fails any sheet whose number disagrees (nbt_pipeline.py:12402-12430, 12558). Judges are told that 0 is 'a real and EXPECTED answer', that cosmetic differences are worth zero, and to 'judge CONTENT, not style' (nbt_pipeline.py:3846-3851). Blinding is thorough: salted per-view file permutation, neutralized metadata, no review findings, no orchestrator scans, no provenance vocabulary (README.md:388-426).

Parameter	Value	Evidence
Scale	Integer -4..+4, fixed rungs with anchors	nbt_pipeline.py:3826-3844
Score derivation	Capped sum of ledger rows; sheet fails if integer disagrees	nbt_pipeline.py:3873-3885, 12402-12430, 12558
Tier order (prompt)	correctness > consistency > preservation > completeness > formatting (writing absent)	nbt_pipeline.py:3899-3901
Tier order (schema)	BASIS_TIERS adds 'writing' as a 6th value; caps +/-1, minor-only	nbt_pipeline.py:1399-1413
Per-comparison coverage	'checks' row for EVERY frozen check id (M1-M17, M18-M24, J1-J4) - about 26 cells per opponent	nbt_pipeline.py:3961, 3974-3984, 10438-10440
Sweep workload	M1-M17 + J1-J4 (+ M18/M19/M20) re-run by the judge on its target; opponents read in full, not swept	nbt_pipeline.py:3769-3783
Anti-regression gate	median of 6 vs-original scores >= 0.0 (rule switchable to sign test)	nbt_pipeline.py:2453-2462, 12983-12984
Aggregate forbidden	No overall/mean/rank field from judges; aggregation is code's job	nbt_pipeline.py:3986-3988

Table 3 - Judge contract essentials

One structural property matters for everything that follows: the panel measures **differences**, not quality. A candidate's median is only meaningful relative to the exact field it was measured in, and the field is rebuilt every round. There is no persistent rating, no cumulative quality estimate, and no requirement that a new champion be better than the round-1 champion in any comparable unit - only that it not regress against the pristine original on six coarse scores. This is the soil in which the convergence problems of chapter 4 grow.

Cross-Agent Criteria Conflicts (Hypothesis 1)

Your first hypothesis - different prompts to codex matching different scoring functions - is correct, and the repo even advertises the opposite: its own design goal is 'one defect vocabulary, one artifact process' so that 'an improvement one session sees is an improvement the panel can score' (README.md:428-464). The mechanism exists (DEFECT_CLASS_RULE is generated from BASIS_TIERS 'so it cannot drift', nbt_pipeline.py:2099-2101) and is genuinely good design. But four conflicts leak around it.

D-01 - The writing rubric swallows semantic defects

WRITING_RUBRIC defines twelve named checks Q1-Q12 and instructs judges to 'use it to justify every writing-tier row (one named check per row)' (nbt_pipeline.py:11551-11572). Q1 is 'the sentence's factual premise is contradicted by the data or by another sentence' and Q2 is 'a formal-logic slip' - these are semantic defects, and by the priority order they belong in the correctness tier, where they would be worth up to +/-4. But the rubric explicitly files them under the writing tier, which is minor-only and capped at +/-1 across ALL writing rows combined (nbt_pipeline.py:1404-1413, 3873-3878). Meanwhile DEFECT_CLASS_RULE tells every session that a category-2 defect goes to correctness 'when the wording changes the meaning'

(nbt_pipeline.py:2063-2076). The same defective sentence is therefore a correctness item if the judge reads it as meaning-changing and a writing item if the judge reads it as prose - a judge-dependent classification with a 4x swing in score consequences. Two judges looking at the same pair can legitimately disagree not about the facts but about the arithmetic class, which is exactly the calibration noise the graded basis was built to remove.

D-02 - The 'cannot drift' vocabulary drifted

The judge prompt's PRIORITY ORDER block lists five tiers: 'correctness > consistency > preservation > completeness > formatting' - writing is absent (nbt_pipeline.py:3899-3901). The schema it must fill has six basis values (BASIS_TIERS, nbt_pipeline.py:1399-1400). DEFECT_CLASS_RULE_TEMPLATE opens with 'Five classes, in this fixed priority order' and then renders six (nbt_pipeline.py:2061, 2101). The consequence is not cosmetic: the graded basis says 'basis must not claim a lower-priority tier than the items you list' (nbt_pipeline.py:3860-3871), but the prompt never defines whether writing ranks above or below formatting, so a judge holding one writing item and one formatting item has no defined way to pick the basis tier. This drift happened inside the very mechanism whose comment says it cannot drift, which is the strongest possible argument for generating every tier listing from a single source, including the judge prompt's hardcoded PRIORITY ORDER block.

D-03 - Em-dash: one rule, four meanings

The em-dash rule is the sharpest test of your hypothesis, so it deserves exactness. What the repo does NOT have is a reviser prompt that favors em-dashes; nothing in nbt-revise or the revision directives encourages them. What it does have is the same rule wearing four different uniforms:

- **Code-side scanner:** FMT-P1 fires when density exceeds max_em_dashes_per_1000 (default 2.0 per 1000 words) with remediation text 'rewrite parenthetical dashes as commas/parentheses' (nbt_docx_format.py:2155-2161, 74).
- **Reviewer:** the M20 sweep seeds em-dash density rows for disposition (nbt_pipeline.py:1833-1848), but FMT-P1 is NOT in FINDING_TIER_RULES (nbt_docx_format.py:595-607) - it is advisory, so the reviewer may close it as 'editorial preference', and the auditor's attack mandate explicitly covers finding-tier rows only, leaving em-dash closures un-audited (nbt_pipeline.py:3136-3143).
- **Reviser:** M20_REVISE_RULE mandates 'Fix the EDITORIAL rows yourself: rewrite parenthetical em-dashes as commas/parentheses' (nbt_pipeline.py:1849-1860) - mandatory effort.
- **Judge:** M20_JUDGE_SWEEP prices formatting uniformity at 'at most +/-1' and never decisive (nbt_pipeline.py:1872-1885), and the writing rubric's Q11 typography list mentions 'a spaced hyphen used as a dash' but not em-dash density (nbt_pipeline.py:11562-11563).

There is also a direct tool-internal contradiction: the FMT-T8b nested-parenthesis finding - which IS finding-tier - carries the remediation advice 'move the inner item out (comma or em dash) or restructure the sentence' (nbt_docx_format.py:1639-1642). An agent that fixes a nested-parenthesis defect by following the scanner's own advice inserts an em dash and can trip FMT-P1, turning one finding into another. The residual-issue ledger itself flags the em-dash cap as 'pipeline/user preference, not journal rules' (NBT_RESIDUAL_ISSUE_LEDGER.md:254, FP-06), which makes the asymmetry starker: an admitted

user preference is mandatory reviser work, invisible audit-wise, and nearly invisible to the panel.

D-04 - The judge is promised differences the normalizer deletes

M20_JUDGE_SWEEP states: 'The TEXT-level consistency rows (citation dialect, US/UK spelling, attributive hyphenation) are deliberately NOT pre-normalized: a corpus that completed a convention where the other did not is a real CONSISTENCY-tier difference (second in the priority order) and is scored as one' (nbt_pipeline.py:1872-1885). This is false on the current tree. FMT-T8a/T8d/T8e are classified 'mechanical' under the default policy (citation_journal_names=drop, term_spelling=dominant, term_hyphenation=dominant; nbt_docx_format.py:56-76, 1923-1949), and the orchestrator runs normalize_formatting_in_dir on every stage package BEFORE it is fingerprinted, pinned, or judged (nbt_pipeline.py:6141-6191, 6236-6260). Two consequences follow. First, judges are invited to hunt a +/-3 consistency difference class that the pipeline mechanically erases - wasted judge attention and a stale contract. Second, and worse: 'dominant' means each package is normalized to ITS OWN majority form, so two candidates whose text has drifted can emerge with DIFFERENT conventions (one US-dominant, one UK-dominant), manufacturing exactly the cross-package inconsistency the judge was told to score. A normalizer that targets a per-package majority is a convention generator, not a convention enforcer.

Defect class	Reviewer / auditor	Reviser / integrator	Judge panel	Net effect
Em-dash density (FMT-P1)	Advisory row; not auditor-attackable	Mandatory fix (M20_REVISE_RULE)	Formatting tier, <= +/-1, minor-only	Effort without reward; audit blind spot
Spelling / hyphenation / citation dialect (T8a/T8d/T8e)	Seeded as findings to dispose	Auto-fixed by normalizer before delivery	Promised as consistency +/-3; actually erased	Judge hunts phantom differences; per-package normalization can create new ones
Premise / logic slip (Q1/Q2)	Category-2 finding; correctness if meaning changes	Fixed via language pass L1-L2	Writing rubric routes to minor-only writing tier (+/-1)	Judge-dependent tier = calibration noise
Grammar / typo (Q10/Q11)	Category-2 minor findings	Language pass L10-L11 mandatory	Writing tier, whole tier shares one point	Repo's own C20 verdict: selection cannot prefer a cleaner version
Auditor adds (AU-*)	Added without independent verification	Must resolve every AU-*; cannot drop	Never sees provenance (blinded)	Asymmetric false-positive economics

Table 4 - Cross-agent conflict matrix (same defect, four pricings)

Convergence Mechanics (Hypotheses 2 and 3)

Non-convergence here has a precise mechanism: **nothing is cumulative**. Each round's statistics are field-relative medians over a freshly composed field (nbt_pipeline.py:12826-12990). The field always contains the pristine original and every earlier pin (nbt_pipeline.py:146-153), so as rounds progress the comparison set changes, the directed-score lists change, and the medians of identical content change with them. The one statistic computed against a fixed anchor - vs_original, the median of six scores - is explicitly 'coarse' (nbt_pipeline.py:2453-2462), and because round 1's base IS the original, every round-2+ arm is derived from a champion that already cleared the gate; the gate thereafter only catches catastrophic regressions, not slow drift. Real quality differences between successive candidates are on the order of one scale rung, while the sampling noise on a $2 * \text{judges} * (\text{field} - 1)$ -value integer median with heterogeneous judge behavior is of the same order. Round outcomes are therefore noise-dominated, and since each round re-measures from scratch, the score trajectory as rounds increase is a random walk around 'slightly better than the original', not a convergent sequence.

The pipeline's own history concedes the measurement problem from the other direction: the comment on the scale notes 'the production panel (294 directed scores, 2026-09-17) used -3..3 and never once used +/-4' (nbt_pipeline.py:1390-1396) - the extreme rungs are dead vocabulary, the effective scale is seven rungs wide, and the anchors admit panels compress into the middle. A median statistic on a compressed integer scale with per-tier caps is a blunt instrument for detecting one-rung improvements; it is, however, a precise instrument for detecting ties, and ties are what the retention rule acts on.

D-07 - The stasis ratchet: 0-expected panels, asymmetric scrutiny, incumbent retention

Three reinforcing mechanisms damp the selection gradient. First, the judge prompt says '0 is a real and EXPECTED answer... an all-zero comparison sheet is valid and is often the honest sheet' and 'do not reward a version merely for being different, newer, longer, shorter, or more heavily edited' (nbt_pipeline.py:3846-3851) - the correct instruction for avoiding fabricated differences, but it biases the panel toward the status quo whenever improvements are subtle. Second, scrutiny is asymmetric: the judge runs the full mechanical sweep set on its TARGET and merely reads each opponent in full (nbt_pipeline.py:3769-3783). Defects in the target are enumerated by scripts; defects in the opponent are found by attention. 'Resolved' items (target fixes what opponent still carries) are therefore systematically under-enumerated relative to 'introduced' items, pressing all scores downward toward 0 and -1 - a pessimistic panel. Third, when the panel cannot distinguish a challenger from the base on (median, mean, IQR), the incumbent is retained by rule (nbt_pipeline.py:13312-13324). Each mechanism is individually defensible; together they form a ratchet - challenger must be decisively better to move, one outlier judge can flip a close pair the other way, and the base wins every ambiguity.

D-08 - Too much scoring: the judge redundancy

The cost structure concentrates where the information is least scarce. Every judge session re-runs the complete frozen sweep set (M1-M17, J1-J4, plus M18/M19/M20) on its target (nbt_pipeline.py:3769-3774)

and then disposes EVERY frozen check id on EVERY comparison - about 26 'checks' cells per opponent, 156 cells per sheet in the default 7-member field, with a missing cell failing the entire run (nbt_pipeline.py:3961, 3974-3984, 10438-10440). The three judges assigned to the same candidate triplicate identical sweep work on identical bytes; nothing is shared between them because blinding forbids seeding 'a pre-computed row [that] would be an answer key' (README.md:460-461). But the orchestrator itself computes exactly those deterministic scans after the fact - postcheck_judge re-scans the blinded target and archives the result (nbt_pipeline.py:12690-12712) - which proves the rows are computable from the blinded view alone and carry no provenance. The design pays triple price for sweeps that are (a) deterministic code output, (b) recomputed anyway, and (c) identical across a candidate's judges, while the genuinely judgmental work - comparing versions - is squeezed into whatever attention remains.

D-09 - The merge-into-a-base strategy is suboptimal, and it is the only accumulator

Integration is where the design concentrates its hopes: $K = 1 + M + N$ integration arms, one per pool member, each reading every other package in full and porting improvements (nbt_pipeline.py:184-187, 3200-3398). Two design choices make it conservative by construction. Class (d) orders 'a defect present in the base AND in at least one donor -> LEAVE UNTOUCHED (do not "improve" the shared majority...)' (nbt_pipeline.py:3246-3249), and conflicts 'PREFER THE BASE's version' (nbt_pipeline.py:3256-3259). An integrator is therefore a superset of its base at best, and typically a near-duplicate of it. The consequences compound: (1) K near-duplicate arms enter the field and split the panel's votes against the arms they were derived from; (2) the pipeline's ONLY mechanism for carrying a losing arm's good idea into the next round is an integration port, and that mechanism is instructed to prefer the base on any ambiguity - so improvements that lose their round are structurally under-absorbed; (3) the cost is $O(K \times |pool|)$ full reads per round, the largest single block of non-judge spend. The user-side intuition 'merging into a base is suboptimal' is thus correct twice over: wrong strategy (many conservative merges instead of fewer, bolder ones) and wrong monopoly (it is the sole accumulation channel).

D-10 - Too much rewriting at the beginning; paraphrase laundering across rounds

Rewrites 'run FIRST in this round, before the review' (nbt_pipeline.py:3417-3419; DAG at 172-177 shows rewrites depend only on a1). In round 1 that means a structural rewrite reorganizes a manuscript that no review has yet examined - the arm's only feedback is its own PROBLEMS SURFACED list, which is explicitly 'NOT a finding list' and 'NOT scored as one' (nbt_pipeline.py:3597-3604). And because the odd-arm convention makes $M=1$ a structural arm, every later round also carries an organization-level churn candidate. Organization churn is the enemy of the prior-round finding reconciliation, which is the pipeline's cross-round memory: the reviewer must mark each prior finding either STILL PRESENT or 'NO LONGER PRESENT (fixed, or the quoted text/location is gone)' (nbt_pipeline.py:10738-10755). The second clause is a text-location test, not a semantic one: a rewrite that paraphrases a defective sentence destroys the quoted evidence, the finding is recorded as not reproducible, it drops out of the frozen list, and the revisers never see it - unless one of the fresh sweeps happens to re-find it. Paraphrase laundering is therefore a first-class non-convergence mechanism at the semantic level, and it interacts perversely with the judge's

under-enumeration bias: the carried-finding ledger leaks exactly where the panel is weakest.

D-11 - All-or-nothing panels can void a round

Eligibility requires a candidate's directed-score panel to be exactly complete (nbt_pipeline.py:13231-13247), and a sheet that omits one issued label or one check cell fails its run (nbt_pipeline.py:12528-12535, 3974-3984). If failures exhaust the retry budget (default 2 retries, 3 attempts, nbt_pipeline.py:103-107), the candidate is ineligible; if enough candidates fall, the round reports 'no eligible rows' and the champion falls back to A1_r - 'this round made no demonstrable, non-regressing progress' (nbt_pipeline.py:13258-13264). The design is principled (a partial panel must not decide a round) but the granularity is wrong: one dropped comparison in one sheet of 21 nullifies the entire round's fresh work (M+N+K full agent sessions), even when every other number is present and the missing pair could not have changed the ranking. A cheaper middle ground - retry only the missing pair's comparison, or admit panels missing non-decisive pairs with explicit disclosure - would remove the cliff without weakening the invariant that a round must be fully measured before it is decided.

29 sessions/round, 21 of them judges

Default plan (M=2, N=1, judges=3, 7-member field). One dropped comparison cell can discard all of it via the base fallback.

Judge vs. Agent Perception Mismatch (Hypothesis 4)

Blinding is the pipeline's proudest feature and it is genuinely rigorous: judges get no review findings, no ledgers, no change records, no orchestrator scans (README.md:388-414). The cost is that the panel and the workers optimize different objective functions. Revisers are graded by the frozen review's finding list and their own revision ledger; integrators by the DIFF_LEDGER port discipline; rewrites by the organization map and problems surfaced. None of that work is visible to a judge except as content differences the judge can personally re-derive. The shared defect vocabulary is supposed to bridge the two worlds ('The same defect must keep the same class from the session that finds it to the session that scores the result', nbt_pipeline.py:2093-2098), and chapter 3 shows exactly where the bridge cracks: tier routing (D-01), vocabulary drift (D-02), and classes the judge is told about but cannot observe because the normalizer erased them (D-04).

A second, subtler mismatch is temporal. A judge reads the target and the opponents as static documents; it has no notion of which defects are NEW relative to the round's base or the original beyond what its own comparison reveals. The anti-regression gate leans on the judge for exactly this: if an opponent IS the pristine original, 'a POSITIVE score means the target improved on the original and a NEGATIVE score means the

target REGRESSED against it. That single comparison becomes the field's anti-regression gate' (nbt_pipeline.py:3910-3927). But the same judge was just told that 0 is expected, that cosmetic differences score zero, and that opponents' defects are found only by plain reading. A judge that under-enumerates the original's defects (it is, after all, just one more document to read) produces a vs-original median near 0 for every candidate, which passes the gate vacuously - the gate exists to catch regressions and a 0-heavy panel lets micro-regressions through in both directions of the same coin.

D-12 - Self-reported tie-breaks are the only agent voice, and they are unverifiable

critical_remaining (CRITICAL findings still open) and writing_remaining (category-2 findings still open) are ranking keys that sit BELOW the panel statistics but ABOVE the content digest (nbt_pipeline.py:13305-13309). Both are self-reported by the producing session in its completion marker. The code is honest about the weakness: 'self-reported by the producing agent (reported and cross-checked, but not verifiable by the panel)', and 'a rewrite or integration self-report is unverified' - the cross-check applies only to the revise arm and only in the reported-zero direction (nbt_pipeline.py:13112-13140). The repo's own design triage examined this (C22, NBT_DESIGN_TRIAGE_LEDGER.md:234-241) and ruled NOT_A_BUG by design, while simultaneously refusing to rank on manual_steps 'because ranking on it would reward under-reporting' (nbt_pipeline.py:13305-13306). The two decisions are inconsistent: an agent that wants to win a tie has the identical incentive to under-report critical_remaining that it has to under-report manual_steps; the only difference is that one number was fenced off and the other was made a ranking key. The panel-verifiable alternative already exists in the same JSON - the graded-basis item counts judges produced - and is simply not used for this purpose.

D-13 - Arithmetic overrides judgment

Contract v3 makes the judge's integer fully derivable: 'code derives the number the rows support and rejects a sheet whose integer contradicts it' (nbt_pipeline.py:12402-12430; enforcement at 12558). This buys calibration and costs agency. A judge whose holistic reading of a pair is 'clearly better' must either find enough ledger rows to arithmetically justify +3 (pressuring row inflation - gravitating toward whatever enumerable defects make the number work) or submit the number its rows support (subordinating judgment to enumeration). The design comment frames this as calibration, and it does prevent unanchored numbers; but combined with the 0-expected norm and the under-enumeration asymmetry, the operational effect is that the panel's information comes almost entirely from what is script-enumerable on the target side. Judges in this system are less like referees and more like auditors of the target against the opponents' prose.

Why Grammar and Semantics Cannot Win (Hypothesis 5)

Your fifth hypothesis - judges ignoring semantic/syntax/grammar errors - is the one the repo has already litigated with itself, and the record is instructive. The design triage's C20 states the claim plainly: 'a version that fixes fifty grammatical errors scores 0 against one that fixes none; formatting is capped at +/-1 and can never be decisive' (NBT_DESIGN_TRIAGE_LEDGER.md:217-225). The verdict was NOT_A_BUG - 'documented design decisions... Changing the rubric, the score arithmetic, the stage authorizations and the review taxonomy is a feature redesign' - with the residual risk admitted: 'selection cannot prefer a grammatically cleaner version'. Judge contract v3 (2026-09-21) then added the writing tier, which looks like a fix but is not: the tier is minor-only and capped at +/-1 in total, shared across every grammar, typography, coherence, redundancy AND premise/logic row (nbt_pipeline.py:1404-1413, 3873-3878, 11551-11572). The arithmetic is unchanged where it matters:

Scenario (target vs one opponent)	Ledger rows	Reachable score
Fixes 50 grammar/typography errors, opponent fixes none	50 resolved writing-minor rows	capped sum = +1 (writing tier cap)
Fixes 50 grammar errors AND all formatting findings	50 writing + N formatting minors	+2 (writing +1, formatting +1) - still cannot reach 'clearly better'
Resolves 2 major consistency defects	2 resolved consistency-major rows	+4 -> clamped to +3 (consistency cap 3)
Resolves one critical correctness defect	1 resolved correctness-critical row	+3, and qualifies for the +4 rung
Fixes 30 grammar errors, introduces 30 new ones	0 net writing rows	0 - the cap erases both directions of real work

Table 5 - What the tier caps make reachable (contract v3 arithmetic)

Two distortions follow. First, a reward ceiling: the entire prose-improvement economy - the mandatory L1-L11 language pass every package-producing stage must run and log row-by-row (nbt_pipeline.py:3658-3680), the M20 editorial fixes, the revisers' category-2 findings - is priced at two points out of a nine-point reachable scale, and cannot ever decide a comparison. Second, a semantic leak: because Q1 (premise) and Q2 (logic slip) live inside that same capped writing rubric, the semantic class of defects is only fully priced when a judge classifies it as correctness under the separate meaning-changes rule - the D-01 ambiguity. The system does not literally ignore grammar and semantics; it prices them at nearly zero and makes their pricing judge-dependent, which from the selection gradient's point of view is the same thing. Note the consistency tier is the exception that proves the design's own logic: convention-completion (acronyms, term families, cross-document consistency) gets +/-3 because it is enumerable and script-checkable (nbt_pipeline.py:3887-3895) - confirming that what the panel actually rewards is what can be mechanically evidenced.

+2 of 9

Maximum reachable advantage from ALL grammar, typography, and formatting work combined (writing cap 1 + formatting cap 1). One critical correctness item is worth +3 alone.

Additional Findings (Open-Ended Sweep)

The auditor stage exists because of a real, measured failure: '96 scan rows were closed with one identical sentence - and the sentences the operator later flagged were inside that pile' (README.md:58-86; NBT_PIPELINE lead, nbt_pipeline.py:3109-3118). Its mandate is asymmetric in three ways. Drops require 'the strongest evidence you can produce - never "I disagree"' and must quote refuting text (nbt_pipeline.py:3120-3130); adds require only the normal finding shape, and nobody verifies them downstream - the revisers 'must resolve' every AU-* finding and 'a dropped finding is NOT yours to re-litigate' (nbt_pipeline.py:4371-4378), so a false AU-* add becomes forced edits with real regression risk, and there is no counterparty whose job is to attack the auditor. Meanwhile the reviser's R1 step still allows discarding ANY finding, including an auditor-confirmed one, with nothing more than 'a concrete recorded rationale' (nbt_pipeline.py:2896-2899) - so confirms are defeatable while drops are locked. And the whole stage is off by default ('The stage is off by default so that existing roots keep their plan shape', README.md:84-86), meaning the documented 96-identical-sentences failure mode remains the default operating mode of the review stage.

D-14 - Judge grounding is warning-only

The panel's premise is that every judgment is grounded in the frozen sweep review, but the enforcement is thin: postcheck_judge requires judge_review/ to exist and be non-empty (nbt_pipeline.py:12718-12724), while the M18/M19/M20 artifact check - 'the judge produced no corresponding artifact table' - is explicitly 'a warning, not a failure: the panel's validity must not depend on an agent's file naming' (nbt_pipeline.py:12690-12712). The repo's own triage (C21) upheld this as a documented bar. The graded-basis `checks` field was added in v3 precisely to close this hole - a disposition for every check id on every comparison - yet nothing cross-validates a 'clean' disposition against the presence of the corresponding sweep artifact, so a judge can declare 26 clean cells per opponent with an inventory file and nothing else. The cheapest fix is not more artifacts but the check that already exists in the sheet: consistency between `checks` values and artifact existence.

D-15 - Operational defaults and miscellany

Three smaller items from the sweep. (1) The default concurrency is 255 simultaneous agent sessions (nbt_pipeline.py:757-770) with a 4-hour per-run timeout and no token or cost governor anywhere in the config surface; on a 7-member field that is 21 concurrent judges each holding several full corpus copies, and the repo notes a three-round run 'is easily >10 GB' (nbt_pipeline.py:17185). A default of jobs=min(8, cpu*2) with an explicit flag for more would fail safe. (2) The judge prompt acknowledges there is 'no OS-level jail' -

a judge that ignores its prompt can walk the pipeline root (README.md:416-420); the mitigations (prompt language + orchestrator-side re-derivation) are reasonable, but the surface remains: package CONTENT itself reaches the judge unfiltered, so text embedded in the manuscript is a prompt-injection channel into the panel (the U1 audit fixed the bookkeeping-file variant; the manuscript-text variant is inherent). (3) The two source hierarchies noted by C36 (skill propagation order vs master-prompt authority order) are now documented in the prompts themselves, which is the right fix, but the duplication remains a drift risk of the same family as D-02.

Root-Cause Synthesis and Recommendations

Every finding above is a symptom of one design stance: the pipeline is built to CONSERVATIVELY SELECT between near-identical versions rather than to ACCUMULATE improvements. Selection wants exactly what this repo built - blinding, capped tiers, 0-expected norms, incumbent retention, anti-regression gates - because all of it suppresses false positives in promotion. But the same machinery taxes every true positive: prose work is capped at +2, losing arms' ideas only survive through a base-biased integration monopoly, cross-round memory is a text-match ledger that rewrites destroy, and 29 sessions per round buy 21 re-measurements of the same corpus instead of new information. The scoring mismatch you hypothesized is real but it is mostly NOT prompt-vs-prompt disagreement; it is a systematic mispricing between what the worker stages are told to do (enumerate, fix, port, log) and what the panel can arithmetically reward (script-evidenced, high-severity, difference-making content). Agents respond to incentives exactly as the ledger warns: under-report what is costly to report, do the work that is scored, and treat the rest as compliance.

Recommendations

ID	Fix (1-2 sentences)	Where
D-01	Move Q1/Q2 out of WRITING_RUBRIC into the correctness tier's named checks (or allow writing rows to carry major/critical severity when meaning changes), so semantic tier routing is deterministic.	nbt_pipeline.py:11551-11572, 1404-1413
D-02	Generate the judge prompt's PRIORITY ORDER block and the 'Five classes' count from BASIS_TIERS like defect_class_rule() does; add a test asserting all three listings match.	nbt_pipeline.py:2061, 2101, 3899-3901
D-03	Rewrite FMT-T8b's remediation text to suggest commas/parentheses only; pick one status for FMT-P1 (either add it to FINDING_TIER_RULES and let the auditor attack it, or drop it from M20_REVISE_RULE's mandatory list).	nbt_docx_format.py:1639-1642, 595-607; nbt_pipeline.py:1849-1860
D-04	Normalize spelling/hyphenation/citation dialect to a FIXED target recorded once at setup (not per-package majority), and re-word M20_JUDGE_SWEEP to match what actually ships to judge views.	nbt_docx_format.py:56-76, 1931-1949; nbt_pipeline.py:1872-1885, 6141-6191
D-05	Require the same evidence shape for AU-* adds as for drops (verbatim quote + location), add a cheap postcheck that re-locates each AU-* quote, and either lock R1 discards of confirmed findings under --audit on or default audit on for the final round.	nbt_pipeline.py:3120-3145, 2896-2899, 4371-4378
D-06	Carry a frozen anchor arm (round-0 champion) in every round's field and report a running vs-anchor trend line in DECISION_REPORT.md; or maintain a simple cross-round rating (e.g. mean directed score vs all pins) as a reported, never gating, trend.	nbt_pipeline.py:146-153, 12826-12990

ID	Fix (1-2 sentences)	Where
D-07	Reduce the scrutiny asymmetry by handing every judge the same neutral code-side scan rows for target AND opponents (deterministic, provenance-free, already computed post-hoc); reserve the opinion sweeps for the reviewer.	nbt_pipeline.py:3769-3783, 12690-12712
D-08	With D-07 in place, cut judge sweeps to the frozen judgment set only, or run 1 sweeping judge + N-1 comparison-only judges per candidate; expected saving: well over half of judge cost at equal panel validity.	nbt_pipeline.py:3769-3784, 10438-10440
D-09	Default the integrator mask to the base plus the panel-best challenger (2 integrations), or seed one integration from the panel's own top-2; allow class (d) ports when ≥ 2 donors independently fixed the same shared defect (evidence of benefit).	nbt_pipeline.py:754, 3240-3260
D-10	Make prior-finding reconciliation semantic (paraphrase-tolerant match or an explicit 'fixed-by-rewording' verdict the reviewer must justify), and consider retiring the structural rewrite arm after round 1 or pointing it at the prior round's PROBLEMS SURFACED.	nbt_pipeline.py:10738-10755, 3417-3419
D-11	Retry the missing comparison, not the whole session; or admit panels that miss non-decisive pairs (both rows present in every OTHER pairing) with an explicit 'partial panel' disclosure in the decision record.	nbt_pipeline.py:13231-13264, 12528-12535
D-12	Replace self-reported tie-break keys with panel-verifiable proxies already in hand (graded-basis resolved/introduced counts per candidate), or extend the zero-direction cross-check to both directions and all arm types.	nbt_pipeline.py:13112-13140, 13305-13309
D-13	Allow a documented +/-1 discretion band around the derived score with a mandatory reason, or keep v3 as is and add one line to the prompt acknowledging the enumeration-over-judgment tradeoff.	nbt_pipeline.py:12402-12430, 12558
D-14	Under --strict-artifacts, fail a sheet whose `checks` say 'clean' for a check whose artifact is absent; otherwise record artifact-vs-disposition mismatches as panel quality metrics.	nbt_pipeline.py:12690-12724, 3974-3984
D-15	Default jobs to a small bound with an explicit opt-up flag; add a per-round session/token cost report to the run record.	nbt_pipeline.py:757-770

Table 6 - Brief fixes, keyed to findings

None of these changes requires touching the content-addressed integrity chain, the blinding machinery, or the round DAG - the genuinely strong parts of this design. The first three (D-01, D-02, D-04) are pure prompt/constant repairs with immediate calibration payoff; D-07/D-08 are the largest cost lever; D-06/D-10 are the convergence levers; and D-05/D-12 close the two incentive holes the repo's own ledgers already spotted but ruled out of scope. If the operator's goal is a pipeline whose round count actually converges toward a measurably better manuscript, the sequencing that matters is: fix the vocabulary drift first (cheap, removes noise), then re-price the tiers (D-01) and the normalizer target (D-04), then restructure judge cost (D-07/D-08), and only then tune the selection dynamics (D-06/D-07/D-09/D-10) - because a cheaper, correctly-priced panel is the prerequisite for any tuning to be measurable.

Appendix A - Evidence Index

All citations point to the repository at commit 7d19d80 (working tree restored from backup-092203-7d19d80.git.zip). Line numbers refer to the files as checked out; nbt_pipeline.py is 17,820 lines, nbt_docx_format.py 3,046+. Prompt constants are assembled at runtime by token substitution, so the rendered prompts contain the cited blocks verbatim.

ID	Artifact / behavior	Citation
-	Judge prompt: scale, graded basis, priority order, schema, completion contract	nbt_pipeline.py:3729-4003
-	WRITING_RUBRIC Q1-Q12; LANGUAGE_PASS_RULE L1-L11	nbt_pipeline.py:11551-11572; 3658-3686
-	BASIS_TIERS, TIER_ROW_WEIGHTS, TIER_CAPS, MINOR_ONLY_TIERS	nbt_pipeline.py:1399-1414
-	DEFECT_CLASS_RULE_TEMPLATE ('Five classes'); generator from BASIS_TIERS	nbt_pipeline.py:2059-2101
D-01	Writing-tier routing of Q1/Q2; minor-only caps	nbt_pipeline.py:11551-11572, 3873-3878, 1404-1413
D-02	Priority order omits writing; template says 'Five classes'	nbt_pipeline.py:3899-3901, 2061, 2101, 1399-1400
D-03	FMT-P1 em-dash rule + default cap; T8b fix text; FINDING_TIER_RULES; M20 rules	nbt_docx_format.py:2155-2161, 74, 1639-1642, 595-607; nbt_pipeline.py:1833-1885
D-04	POLICY_DEFAULTS; mechanical rule classes; normalizer before judging; M20_JUDGE_SWEEP claim	nbt_docx_format.py:56-76, 1923-1949; nbt_pipeline.py:6141-6191, 6236-6260, 1872-1885
D-05	AUDIT_DIRECTIVES; reviser audit block; revise R1 discard rule	nbt_pipeline.py:3092-3197, 4371-4378, 2896-2899
D-06	Field composition; aggregation; vs-original gate; scale history note	nbt_pipeline.py:146-153, 12826-12990, 2453-2462, 1390-1396
D-07	0-expected norm; target-only sweeps; incumbent retention; vs_base not a key	nbt_pipeline.py:3846-3851, 3769-3783, 13312-13324, 13297-13304
D-08	Judge sweep mandate; per-comparison checks; JUDGE_COVERAGE_CHECKS; postcheck re-scan	nbt_pipeline.py:3769-3784, 3961-3984, 10438-10440, 12690-12712
D-09	Integration classes (a)-(e), prefer-base rule, donor-read mandate; DAG position	nbt_pipeline.py:3240-3260, 3200-3398, 184-187
D-10	Rewrite-first placement; prior-round reconciliation rule; PROBLEMS SURFACED status	nbt_pipeline.py:3417-3419, 172-177, 10738-10755, 3597-3604
D-11	Eligibility/panel completeness; base fallback; retry policy; sheet omission failure	nbt_pipeline.py:13231-13264, 103-127, 12528-12535
D-12	Tie-break inputs and cross-check scope; ranking key; manual_steps exclusion rationale	nbt_pipeline.py:13112-13140, 13305-13309
D-13	derived_comparison_score; enforcement of derived != score	nbt_pipeline.py:12402-12430, 12558
D-14	postcheck_judge warning-only grounding; judge_review existence gate	nbt_pipeline.py:12690-12724

ID	Artifact / behavior	Citation
D-15	DEFAULTS (jobs, timeout, retries); disk-footprint note	nbt_pipeline.py:757-770, 17185
-	Repo's own verdicts on C20/C21/C22 (kept by design)	NBT_DESIGN_TRIAGE_LEDGER.md:217-241
-	FP-06 em-dash cap acknowledged as user preference	NBT_RESIDUAL_ISSUE_LEDGER.md:249-256

Table 7 - Primary sources by finding

Appendix B - Verification of the Original Hypotheses

Hypothesis (as submitted)	Verdict	What the repo actually does
Different prompts to codex match different scoring functions (judges penalize em-dash; reviser favors em-dash; auditor and reviewer use different agendas)	PARTIAL	Four overlapping but misaligned scoring functions exist (Table 4). The literal em-dash direction is refuted - no prompt favors em-dashes; instead the reviser is MANDATED to remove them while the auditor cannot attack the row and the judge caps it at +/-1 (D-03). Auditor-vs-reviewer agenda mismatch confirmed with asymmetric economics (D-05).
Non-convergence of scores as the number of rounds increases	CONFIRMED	Nothing accumulates: field-relative medians, fresh noise per round, no cross-round metric, coarse 6-score vs-original gate that stops binding after round 1 (D-06).
Slower-than-expected convergence (too much scoring; too many rewrites early/late; merge-into-base suboptimal)	CONFIRMED	21 judge sessions/round re-running full sweeps with 3x triplication and no shared evidence (D-08); rewrites precede the review in every round including round 1 (D-10); K=1+M+N conservative, base-biased integrations are the only accumulation channel (D-09); all-or-nothing panels can void rounds (D-11); adaptive stop is opt-in (D-15 context).
Mismatch between what judges perceive as better/worse and what other agents perceive	CONFIRMED	Blinded panel vs finding-driven workers; shared vocabulary drifts (D-02); tier routing is judge-dependent (D-01); judges are promised classes the normalizer erased (D-04); self-reported tie-breaks are the only agent voice and are unverifiable for 2 of 3 arm types (D-12).
Judges ignoring semantic/syntax/grammar errors	CONFIRMED (by arithmetic)	Grammar/prose capped at +/-1 total (writing tier), formatting +/-1, neither can be decisive; repo's own C20 verdict admits selection cannot prefer a cleaner version; semantic defects (Q1/Q2) are double-routed between writing (+/-1) and correctness (+/-4) (D-01, chapter 6).

Table 8 - Hypothesis verdicts

Reading guidance: PARTIAL on hypothesis 1 means the specific em-dash direction inverts in this tree while the underlying point - one defect, different prices per agent - survives in a stronger form. CONFIRMED on hypothesis 5 means the repo prices these defects near zero by design and knows it (C20); for selection purposes that is equivalent to ignoring them, and chapter 8 sequences the repairs by leverage.